This is a comprehensive, single-file React application designed specifically for interview preparation. It combines the most frequently asked-about hooks and patterns into one functional dashboard.

You can copy and paste this directly into a file (e.g., `App.tsx`) in a standard React + TypeScript environment (like Create React App or Vite).

The "React Interview Study Guide" App

This app simulates a dashboard that includes:

1. **Global Theme Management** (Context API)
2. **Complex State Logic** (Reducers)
3. **Performance Optimization** (Memoization)
4. **DOM Manipulation** (Refs)
5. **Side Effects** (Effects)

TypeScript

```
import React, {
  useState,
  useEffect,
  useContext,
  useReducer,
  useRef,
  useMemo,
  useCallback,
  createContext,
  ReactNode
} from 'react';

// --- 1. CONTEXT SETUP (Global State) ---
// usage: Avoids prop-drilling; great for themes or auth state.

type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme>('light');

// --- 2. REDUCER SETUP (Complex State) ---
// usage: Best for complex state logic or when next state depends on previous state.

interface Todo {
  id: number;
  text: string;
}

type TodoAction =
  | { type: 'ADD'; payload: string }
  | { type: 'REMOVE'; payload: number };

const todoReducer = (state: Todo[], action: TodoAction): Todo[] => {
  switch (action.type) {
    case 'ADD':
      return [...state, { id: Date.now(), text: action.payload }];
```

```
    case 'REMOVE':
      return state.filter(todo => todo.id !== action.payload);
    default:
      return state;
  }
};

// --- 3. CHILD COMPONENT (Performance) ---
// usage: Wrapped in React.memo to prevent unnecessary re-renders.

interface ExpensiveComponentProps {
  calculateValue: () => number;
}

const ExpensiveComponent = React.memo(({ calculateValue }: ExpensiveComponentProps) => {
  console.log('ExpensiveComponent rendered'); // Check console to verify memoization
  return (
    <div style={{ padding: '10px', border: '1px dashed gray', marginTop: '10px' }}>
      <strong>Expensive Child Component</strong>
      <p>Result: {calculateValue()}</p>
    </div>
  );
});

// --- MAIN APPLICATION COMPONENT ---

export default function StudyGuideApp() {
  // A. useState: Simple state management
  const [count, setCount] = useState<number>(0);
  const [theme, setTheme] = useState<Theme>('light');
  const [inputValue, setInputValue] = useState<string>('');

  // B. useReducer: Managing the Todo list array
  const [todos, dispatch] = useReducer(todoReducer, []);

  // C. useRef: Direct DOM access (focusing input) & values that persist without re-renders
  const inputRef = useRef<HTMLInputElement>(null);
  const renderCount = useRef<number>(0);

  // D. useEffect: Side effects (syncing title, data fetching, subscriptions)
  useEffect(() => {
    document.title = `Count: ${count} | Theme: ${theme}`;
    renderCount.current += 1;
  }, [count, theme]); // Dependency array: only runs when count or theme changes

  // E. useMemo: Memoizing expensive calculations
  // usage: Prevents 'heavyCalculation' from running on every render, only when 'count' changes.
  const heavyCalculation = useMemo(() => {
    return count * 1000; // Imagine this was a slow loop
  }, [count]);
```

```
// F. useCallback: Memoizing functions
// usage: Ensures this function reference stays the same unless 'count' changes.
// Crucial when passing functions to React.memo components to prevent them from breaking memoization.
const getCalculation = useCallback(() => {
  return heavyCalculation;
}, [heavyCalculation]);

// Handler functions
const handleAddTodo = () => {
  if (inputValue.trim()) {
    dispatch({ type: 'ADD', payload: inputValue });
    setInputValue('');
    inputRef.current?.focus(); // Using ref to focus input
  }
};

const toggleTheme = () => {
  setTheme(prev => prev === 'light' ? 'dark' : 'light');
};

// Styles for the theme
const appStyles = {
  backgroundColor: theme === 'light' ? '#ffffff' : '#333333',
  color: theme === 'light' ? '#000000' : '#ffffff',
  minHeight: '100vh',
  padding: '20px',
  fontFamily: 'Arial, sans-serif',
  transition: 'all 0.3s ease'
};

return (
  <ThemeContext.Provider value={theme}>
    <div style={appStyles}>
      <h1>React Interview Study Guide</h1>

      <section>
        <h2>1. State & Effects</h2>
        <p>Count: {count}</p>
        <button onClick={() => setCount(c => c + 1)}>Increment</button>
        <button onClick={toggleTheme}>Toggle Theme</button>
        <small style={{ display: 'block', marginTop: '5px' }}>
          (Check window title to see useEffect in action)
        </small>
      </section>

      <hr />

      <section>
        <h2>2. Refs & Reducers (Todo List)</h2>
        <div style={{ display: 'flex', gap: '10px' }}>
          <input
```

```
          ref={inputRef}
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          placeholder="Add a todo..."
        />
        <button onClick={handleAddTodo}>Add</button>
      </div>
      <ul>
        {todos.map(todo => (
          <li key={todo.id}>
            {todo.text}
            <button
              onClick={() => dispatch({ type: 'REMOVE', payload: todo.id })}
              style={{ marginLeft: '10px', color: 'red' }}
            >
              X
            </button>
          </li>
        ))}
      </ul>
      <p>Component Render Count (useRef): {renderCount.current}</p>
    </section>

    <hr />

    <section>
      <h2>3. Memoization (useMemo & useCallback)</h2>
      <p>Heavy Calculation Result (useMemo): {heavyCalculation}</p>

      {/* We pass a function to the child. If we didn't use useCallback,
          this child would re-render every time the parent rendered. */}
      <ExpensiveComponent calculateValue={getCalculation} />
    </section>
  </div>
</ThemeContext.Provider>
);
}
```

Study Breakdown: How to explain this in an interview

Here is what each section achieves and the key concepts you should mention if asked about them.

1. State Management (`useState` & `useReducer`)

- **Concept:** React components render based on state.
- **The Code:** We use `useState` for simple primitives (like `count` or `theme`) and `useReducer` for the Todo list.
- **Interview Tip:** Use `useReducer` when state logic is complex (multiple sub-values) or when the next state depends on the previous one in complicated ways. It is similar to Redux but built-in.

2. Side Effects (useEffect)

- **Concept:** Synchronizing the component with external systems (DOM, API, subscriptions).
- **The Code:** We update document.title whenever count or theme changes.
- **Interview Tip:** Always mention the **Dependency Array** (the second argument []).
    - []: Runs only once on mount.
    - [prop]: Runs on mount and when prop changes.
    - No array: Runs on *every* render (dangerous).

3. References (useRef)

- **Concept:** A container for a mutable value that does **not** cause a re-render when changed.
- **The Code:**
    1. inputRef: Accesses the actual DOM node to call .focus().
    2. renderCount: Tracks how many times the app rendered without triggering an infinite loop of re-renders.

4. Memoization (useMemo & useCallback)

- **Concept:** Performance optimization.
- **The Code:**
    - useMemo: Caches the **result** of a function (count * 1000). It only recalculates if dependencies change.
    - useCallback: Caches the **function definition** itself. This is vital when passing functions as props to memoized child components (ExpensiveComponent).
- **Interview Tip:** Don't optimize prematurely. Use these only when operations are expensive or when referential equality is required (like in dependency arrays).

5. Context API (createContext & useContext)

- **Concept:** Avoiding "Prop Drilling" (passing data through many layers of components).
- **The Code:** We create a ThemeContext so any component in the tree could theoretically access the current theme without having it passed down manually.

Next Step

Would you like me to add a **Custom Hook** to this file (e.g., useWindowSize or useFetch) to demonstrate how to extract logic for code reusability?

Here is the updated **single-file** TypeScript React app. I have added a **Custom Hook** (useFetch) to simulate fetching data from an API.

This is a very common interview task: "Write a hook that handles loading, error, and success states."

The "React Interview Study Guide" App (v2)

TypeScript

```
import React, {
```

```
  useState,
  useEffect,
  useContext,
  useReducer,
  useRef,
  useMemo,
  useCallback,
  createContext
} from 'react';

// --- 1. CUSTOM HOOK (Logic Extraction) ---
// usage: Encapsulates async logic so it can be reused in any component.
// Interview Tip: Custom hooks must start with "use" to let React check for rule violations.

interface FetchState<T> {
  data: T | null;
  loading: boolean;
  error: string | null;
}

function useFetch<T>(url: string): FetchState<T> {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // Reset state on url change
    setLoading(true);
    setError(null);

    // MOCKING an API call with setTimeout to avoid external dependencies in this file.
    // In a real interview, you would use: fetch(url).then(res => res.json())...
    const timer = setTimeout(() => {
      // Simulate a success scenario
      if (url === '/api/user') {
        setData({ name: 'Interview Candidate', role: 'Frontend Engineer' } as unknown as T);
        setLoading(false);
      }
      // Simulate an error scenario
      else {
        setError('404 Not Found');
        setLoading(false);
      }
    }, 1500); // 1.5 second delay

    // Cleanup function: Called if component unmounts or url changes before fetch finishes
    return () => clearTimeout(timer);
  }, [url]);

  return { data, loading, error };
}
```

```
// --- 2. CONTEXT & REDUCER SETUP ---

type Theme = 'light' | 'dark';
const ThemeContext = createContext<Theme>('light');

interface Todo { id: number; text: string; }
type TodoAction = { type: 'ADD'; payload: string } | { type: 'REMOVE'; payload: number };

const todoReducer = (state: Todo[], action: TodoAction): Todo[] => {
  switch (action.type) {
    case 'ADD': return [...state, { id: Date.now(), text: action.payload }];
    case 'REMOVE': return state.filter(todo => todo.id !== action.payload);
    default: return state;
  }
};

// --- 3. MEMOIZED CHILD COMPONENT ---

const ExpensiveComponent = React.memo(({ calculateValue }: { calculateValue: () => number }) => {
  console.log('ExpensiveComponent rendered');
  return (
    <div style={{ padding: '10px', border: '1px dashed gray', marginTop: '10px' }}>
      <strong>Expensive Child</strong>: {calculateValue()}
    </div>
  );
});

// --- MAIN COMPONENT ---

interface UserData {
  name: string;
  role: string;
}

export default function StudyGuideApp() {
  // Standard Hooks
  const [count, setCount] = useState<number>(0);
  const [theme, setTheme] = useState<Theme>('light');
  const [inputValue, setInputValue] = useState<string>('');
  const [todos, dispatch] = useReducer(todoReducer, []);
  const inputRef = useRef<HTMLInputElement>(null);
  const renderCount = useRef<number>(0);

  // Custom Hook Usage
  // We simulate fetching a user. Try changing the URL string to force an error.
  const { data: user, loading, error } = useFetch<UserData>('/api/user');

  useEffect(() => {
    document.title = `Count: ${count}`;
    renderCount.current += 1;
```

```
});

const heavyCalculation = useMemo(() => count * 1000, [count]);
const getCalculation = useCallback(() => heavyCalculation, [heavyCalculation]);

const handleAddTodo = () => {
  if (inputValue.trim()) {
    dispatch({ type: 'ADD', payload: inputValue });
    setInputValue('');
    inputRef.current?.focus();
  }
};

const appStyles = {
  backgroundColor: theme === 'light' ? '#ffffff' : '#333333',
  color: theme === 'light' ? '#000000' : '#ffffff',
  minHeight: '100vh',
  padding: '20px',
  fontFamily: 'Arial, sans-serif',
  transition: 'all 0.3s ease'
};

return (
  <ThemeContext.Provider value={theme}>
    <div style={appStyles}>
      <h1>React Interview Study Guide v2</h1>

      {/* SECTION 1: CUSTOM HOOK */}
      <section style={{ border: '1px solid #ccc', padding: '10px', marginBottom: '20px', borderRadius: '8px' }}>
        <h2>1. Custom Hook (useFetch)</h2>
        <p>Status:
          {loading && <span style={{color: 'orange'}}> Loading...</span>}
          {error && <span style={{color: 'red'}}> {error}</span>}
          {user && <span style={{color: 'green'}}> Success</span>}
        </p>
        {user && (
          <div>
            <p>User: <strong>{user.name}</strong></p>
            <p>Role: <em>{user.role}</em></p>
          </div>
        )}
      </section>

      {/* SECTION 2: STATE & EFFECTS */}
      <section>
        <h2>2. State & Effects</h2>
        <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
        <button onClick={() => setTheme(prev => prev === 'light' ? 'dark' : 'light')} style={{marginLeft: '10px'}}>
          Toggle Theme
        </button>
      </section>
```

```
      <hr />

      {/* SECTION 3: REFS & REDUCER */}
      <section>
        <h2>3. Refs & Reducers</h2>
        <div style={{ display: 'flex', gap: '10px', marginBottom: '10px' }}>
          <input
            ref={inputRef}
            value={inputValue}
            onChange={(e) => setInputValue(e.target.value)}
            placeholder="Add task..."
          />
          <button onClick={handleAddTodo}>Add</button>
        </div>
        <ul>
          {todos.map(t => (
            <li key={t.id}>
              {t.text}
              <button onClick={() => dispatch({type: 'REMOVE', payload: t.id})} style={{marginLeft: '10px', color:
'red'}}>x</button>
            </li>
          ))}
        </ul>
        <small>Render Count (useRef): {renderCount.current}</small>
      </section>

      <hr />

      {/* SECTION 4: MEMOIZATION */}
      <section>
        <h2>4. Memoization</h2>
        <ExpensiveComponent calculateValue={getCalculation} />
      </section>
    </div>
  </ThemeContext.Provider>
);
}
```

New Concept: Custom Hooks

If you are asked "How do you share logic between two components that have different UI?" the answer is **Custom Hooks**.

1. **Naming Convention:** You MUST start the function name with `use` (e.g., `useFetch`, `useLocalStorage`). This tells React to enforce rules of hooks (like not calling them inside loops).
2. **Generics (`<T>`):** Notice `useFetch<T>`. This allows the hook to be type-safe regardless of what kind of data you are fetching (a User, a Product, or a List).
3. **State Isolation:** Every time you call `useFetch`, it creates a *new* instance of state. If you used this hook in two different components, they would have their own independent `loading` and `data` states.

Next Step

To practice **Performance**, would you like me to explain (and demonstrate) the specific difference between `useMemo` and `useCallback`? Many candidates mix these two up.

It is very common for interviewers to ask: *"What is the difference between `useMemo` and `useCallback`?"*

Here is the most concise way to remember it:

- **`useMemo`** caches a **value** (the result of a function).
- **`useCallback`** caches the **function itself**.

1. The Analogy

Imagine you have a machine that processes raw diamonds.

- **`useMemo` (The Vault):** You put a raw diamond in, the machine cuts it, and you store the **finished gem** in a vault. Next time you need it, you just grab the gem from the vault instead of cutting it again.
- **`useCallback` (The Tool):** You store the **cutting machine** itself. You aren't storing the result; you are ensuring you don't accidentally buy a brand new machine every day. You keep the same reliable machine ready to be used.

2. The Code Demonstration

Add this snippet to your study file to see the difference in action.

TypeScript

```
import React, { useState, useMemo, useCallback } from 'react';

export default function MemoVsCallback() {
  const [count, setCount] = useState(0);
  const [darkTheme, setDarkTheme] = useState(false);

  // --- useMemo Example ---
  // PROBLEM: This loop is slow. We don't want it running every time we toggle the theme.
  // SOLUTION: useMemo runs ONLY when 'count' changes.
  const expensiveNumber = useMemo(() => {
    console.log('🔄 Calculating expensive number...');
    let total = 0;
    for (let i = 0; i < 1000000000; i++) { total += i; } // Artificial delay
    return count * 2; // Returns a VALUE (number)
  }, [count]);

  // --- useCallback Example ---
  // PROBLEM: Every time this component renders, 'printCount' is technically a *new* function.
  // If we passed this function to a child component, that child would re-render unnecessarily.
  // SOLUTION: useCallback keeps the function reference stable unless 'count' changes.
  const printCount = useCallback(() => {
    console.log('Count is:', count); // Returns the FUNCTION ITSELF
  }, [count]);
```

```
  const styles = {
    backgroundColor: darkTheme ? '#333' : '#FFF',
    color: darkTheme ? '#FFF' : '#333',
    padding: '2rem'
  };

  return (
    <div style={styles}>
      <h2>useMemo vs useCallback</h2>

      {/* Changing theme triggers a re-render, but expensiveNumber is NOT recalculated */}
      <button onClick={() => setDarkTheme(prev => !prev)}>Toggle Theme</button>

      {/* Changing count triggers re-calculation */}
      <button onClick={() => setCount(c => c + 1)}>Increment Count</button>

      <p>Expensive Value: {expensiveNumber}</p>
      <button onClick={printCount}>Log Count</button>
    </div>
  );
}
```

3. Key Interview Questions & Answers

Q: When should I use them?

- **A:** Do **not** use them by default. Only use them when:
  1. You have a computationally expensive calculation (useMemo).
  2. You are passing a function as a prop to a child component wrapped in `React.memo` (useCallback).
  3. You are using a function or object as a dependency in a `useEffect` array (useCallback/useMemo).

Q: What happens if I use them everywhere?

- **A:** It actually hurts performance. React has to allocate memory to store the dependencies and compare them on every render. For simple variables and functions, the cost of comparison is higher than the cost of just re-creating them.

Next Step

Would you like to cover **"Lifting State Up"** vs. **"Prop Drilling"**? This is usually the design pattern question that follows the hooks questions.

"Lifting State Up" is the standard React pattern for sharing data between components. "Prop Drilling" is the messy side effect of doing it poorly (or just deeply).

Here is the breakdown and the code to demonstrate it.

1. The Concepts

- **Lifting State Up:** When two distinct components need access to the same data, you move that state up to their closest common ancestor (parent). The parent then passes the state down to both children.
- **Prop Drilling:** The act of passing data through several layers of components (Parent -> Child -> Grandchild -> Great-Grandchild) just to get it to the bottom. It makes code hard to maintain.

2. The Code Demonstration

Add this section to your study app. This example has two "Sibling" components. They cannot talk to each other directly. They must communicate through the `Parent`.

TypeScript

```
import React, { useState } from 'react';

// --- SIBLING A: The Controller ---
// This component receives a FUNCTION to update the state
const TemperatureInput = ({ onTemperatureChange }: { onTemperatureChange: (temp: string) => void }) => {
  return (
    <fieldset style={{ padding: '10px', marginBottom: '10px' }}>
      <legend>Enter Temperature (Celsius):</legend>
      <input
        type="number"
        onChange={(e) => onTemperatureChange(e.target.value)}
      />
    </fieldset>
  );
};

// --- SIBLING B: The Display ---
// This component receives the VALUE of the state
const BoilingVerdict = ({ celsius }: { celsius: number }) => {
  if (isNaN(celsius)) return <p>Please enter a number.</p>;

  return (
    <div style={{
      padding: '10px',
      backgroundColor: celsius >= 100 ? '#ffcccc' : '#ccffcc',
      fontWeight: 'bold'
    }}>
      {celsius >= 100
        ? "The water would boil."
        : "The water would not boil."}
    </div>
  );
};

// --- PARENT: "Lifting State Up" happens here ---
export default function LiftingStateExample() {
  // 1. The state lives here (the Common Ancestor)
  const [temperature, setTemperature] = useState<string>('');
```

```
  return (
    <div style={{ border: '2px solid #333', padding: '20px', margin: '20px' }}>
      <h3>Parent Component</h3>
      <p>I hold the state, but my children use it.</p>

      {/* 2. Pass the "Setter" to Child A */}
      <TemperatureInput onTemperatureChange={setTemperature} />

      {/* 3. Pass the "Value" to Child B */}
      <BoilingVerdict celsius={parseFloat(temperature)} />
    </div>
  );
}
```

3. Interview Talking Points

Q: Why not just keep the state in the Input component?

- **A:** Because the `BoilingVerdict` component needs to know the temperature to display the message. Sibling components cannot pass data sideways. You MUST lift the state up to the parent so it can flow down to both.

Q: How do you fix Prop Drilling if it gets too deep (e.g., 5 layers down)?

- **A:**
  1. **Component Composition:** Pass the child component itself as a prop (often called `children` or render props), so the intermediate components don't need to know about the data.
  2. **Context API:** If the data is "global" (like theme, user, language), use Context to teleport data to the bottom of the tree.
  3. **State Management Libraries:** For complex apps, use Redux, Zustand, or Recoil.

4. Summary for the Interview

"I lift state up when I need to synchronize two components. If I find myself passing props through 3+ layers of components that don't use the data, I solve that 'Prop Drilling' by introducing Context or using Component Composition."

Next Step

We have covered Hooks, State, Performance, and Data Flow. Would you like to review **Error Boundaries** (handling crashes) or **React Router** (handling navigation)?

React Router is the standard routing library for React applications. It enables you to handle **client-side routing**, meaning your application can update the URL and render different content without making another request to the server for a new HTML document. This is essential for creating seamless Single Page Applications (SPAs).

Here is a review of the core concepts, components, and modern patterns (specifically focusing on **v6** and newer).

---

1. The Core Philosophy

React Router subscribes to the philosophy of **Dynamic Routing**. Unlike traditional static routing (where routes are declared in a configuration file before the app renders), React Router routing takes place **as your app is rendering**.

2. Primary Components

These are the building blocks you will use most often.

- BrowserRouter:
  This is the parent component that wraps your entire application. It uses the HTML5 History API to keep your UI in sync with the URL.
- Routes (formerly Switch):
  This component looks through all its children <Route> elements and renders the one that matches the current URL. In v6, this replaced Switch and includes an intelligent ranking algorithm to pick the best match, not just the first match.
- Route:
  Defines the mapping between a URL path and a component.
  - path: The URL pattern (e.g., /about).
  - element: The React component to render (e.g., <About />).
- Link:
  Used to navigate between pages. It renders an accessible <a> tag but prevents the default browser refresh, handling the navigation internally.
- NavLink:
  A special version of <Link> that knows whether it is "active" (matching the current URL). Useful for navigation menus to style the selected tab.

3. Essential Hooks

React Router relies heavily on hooks to access routing information within your components.

- **useNavigate:** Returns a function that lets you navigate programmatically (e.g., after a form submission).
- JavaScript

```
const navigate = useNavigate();
navigate('/dashboard');
```

-
-
- **useParams:** Returns an object of key/value pairs of the dynamic params from the current URL (e.g., matching /users/:id).
- JavaScript

```
const { id } = useParams();
```

-
-
- **useLocation:** Returns the current location object, which represents the current URL. Think of it like window.location.

4. Basic Implementation Example

Here is a standard setup using **React Router v6**:

```javascript
import React from 'react';
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// 1. Define simple page components
const Home = () => <h2>Home Page</h2>;
const About = () => <h2>About Page</h2>;
const UserProfile = () => {
  // Example of using URL parameters
  const { userId } = useParams(); // Imports needed
  return <h2>User Profile for ID: {userId}</h2>;
};

function App() {
  return (
    // 2. Wrap app in BrowserRouter
    <BrowserRouter>
      <nav>
        {/* 3. Use Link for navigation */}
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      {/* 4. Define Route hierarchy */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        {/* Dynamic Route Parameter */}
        <Route path="/users/:userId" element={<UserProfile />} />
      </Routes>
    </BrowserRouter>
  );
}
```

5. Modern Features (v6.4+ Data APIs)

React Router recently introduced "Data Router" APIs, which allow you to decouple data fetching from rendering. This prevents "request waterfalls" and makes your app feel much faster.

- **Loaders:** Functions defined on a route that run *before* the component renders to fetch data.
- **Actions:** Functions defined on a route to handle form submissions and mutations.

This shifts React Router from being just a UI library to a **framework** that handles data lifecycle management.

Summary Table

Component/Hook	Purpose
<BrowserRouter>	Enables routing using the History API.
<Routes>	Groups routes and chooses the best match.
<Route>	Maps a specific path to a specific Component.
<Outlet>	Used in nested routes to render child content.
useParams	Accesses dynamic URL segments (e.g., :id).
useNavigate	Navigates the user programmatically.

Would you like to see a code example of how to implement Protected Routes (e.g., requiring a login to view a page)?

—---------- SIMPLE REACT APP —----------

Here is a complete, single-file TypeScript React application designed for study. It covers the "Holy Trinity" of React interviews: **State**, **Effects**, and **Data Fetching**.

The Study App: User Directory

This app fetches a list of users from a public API when it loads, handles the loading state, and displays errors if the network fails.

TypeScript

```
import React, { useState, useEffect } from 'react';

// --- SECTION 1: Type Definitions ---
// TypeScript requires us to define the "shape" of our data.
// This interface prevents bugs, like trying to access user.adress instead of user.address.
interface User {
  id: number;
  name: string;
  email: string;
  website: string;
}

const UserDirectory: React.FC = () => {
  // --- SECTION 2: State Management (useState) ---
  // We need three pieces of state to handle a real-world data request:
  // 1. users: Stores the actual data (an array of User objects).
  // 2. loading: Boolean to toggle a "Loading..." message.
  // 3. error: String to store any error messages if the fetch fails.
  const [users, setUsers] = useState<User[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
```

```
// --- SECTION 3: The Side Effect (useEffect) ---
// This hook handles the data fetching.
// It runs AFTER the component renders.
useEffect(() => {
  // We define the async function *inside* the effect because
  // useEffect itself cannot be async (it must return a cleanup function or nothing).
  const fetchUsers = async () => {
    try {
      // Reset error state before starting
      setError(null);

      // Native fetch API
      const response = await fetch('https://jsonplaceholder.typicode.com/users');

      // Check for 404s or 500s (fetch doesn't throw errors for these automatically)
      if (!response.ok) {
        throw new Error(`Error: ${response.status}`);
      }

      const data = await response.json();

      // Update state with the fresh data
      setUsers(data);
    } catch (err) {
      // Type narrowing: ensure 'err' is actually an Error object
      if (err instanceof Error) {
        setError(err.message);
      } else {
        setError('An unknown error occurred');
      }
    } finally {
      // This runs whether we succeed or fail, ensuring the loading spinner stops.
      setLoading(false);
    }
  };

  fetchUsers();

// The dependency array is EMPTY ([]).
// This tells React: "Only run this effect ONCE, when the component mounts."
}, []);

// --- SECTION 4: Conditional Rendering ---
// Before rendering the main UI, we check if we are loading or have an error.

if (loading) {
  return <div className="p-4">Loading users...</div>;
}

if (error) {
  return <div className="p-4 text-red-500">Failed to load: {error}</div>;
```

```
  }

  // --- SECTION 5: The UI Render ---
  // If we get here, we have data. We map over the `users` array to create elements.
  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h1>User Directory</h1>

      <ul style={{ listStyle: 'none', padding: 0 }}>
        {users.map((user) => (
          // The 'key' prop is crucial for React's performance in lists.
          <li
            key={user.id}
            style={{
              marginBottom: '10px',
              padding: '10px',
              border: '1px solid #ddd',
              borderRadius: '4px'
            }}
          >
            <strong>{user.name}</strong>
            <br />
            <small>{user.email}</small>
          </li>
        ))}
      </ul>
    </div>
  );
};

export default UserDirectory;
```

Interview Concepts to Review from this Code

1. **Why use `useEffect`?**
    - **Concept:** React components should be "pure" (input props → output JSX). Data fetching is a "side effect" (it reaches outside the component). `useEffect` isolates this logic so it doesn't block the UI rendering.
2. **The Dependency Array `[]`**
    - **Question:** What happens if you remove the `[]` at the end of `useEffect`?
    - **Answer:** The effect will run after **every** render. Since `setUsers` triggers a re-render, this will cause an **infinite loop** (Fetch → Set State → Re-render → Fetch...).
3. **Async inside `useEffect`**
    - **Question:** Why didn't we write `useEffect(async () => { ... })`?
    - **Answer:** `useEffect` expects its callback to return either nothing or a cleanup function. An async function returns a Promise, which React doesn't know how to handle as a cleanup function.
4. **Handling "Loading" and "Error"**

- **Concept:** This is often a "gotcha" in system design interviews. Never assume the API always works immediately. Always show the interviewer you are thinking about the "Loading" state and the "Error" state.

Would you like me to modify this code to show how to "clean up" a `useEffect` (e.g., cancelling the API call if the user leaves the page)?